# Cahier des Charges : Application de Gestion de Transport

---

## 1. Contexte et Objectifs du Projet

Le projet concerne le développement d'une application complète de gestion de transport englobant la réservation, le suivi et la gestion des trajets. L'application repose sur une architecture client-serveur hybride et distribuée.
Le serveur d'application principal exécutera Node.js sous un environnement Windows, tandis que le serveur de base de données sera hébergé de manière distante sur un système Linux.

## 2. Architecture Technique

### 2.1 Serveur D'Application (Backend)

- **Technologie principale :** Node.js avec le framework Express.js.
- **Système d'exploitation :** Windows Server.
- **API REST :** Exposition d'API sécurisées pour la gestion des trajets, des utilisateurs et des véhicules.
- **Authentification :** Mise en œuvre de jetons JWT (JSON Web Tokens) ou du protocole OAuth2.
- **Journalisation :** Système centralisé de logging des requêtes et événements serveur.
- **Sécurité applicative :** Configuration CORS, chiffrement HTTPS et validation systématique des données d'entrée.

### 2.2 Base de Données

- **Système d'exploitation :** Linux (Ubuntu Server).
- **SGBD :** PostgreSQL ou MySQL.
- **Accès réseau :** Connexion sécurisée à distance via tunnel SSH ou VPN.
- **Structure des données :**

| Nom de la Table | Rôle et Description |
|---|---|
| **Utilisateurs** | Stockage des comptes (clients, chauffeurs, administrateurs) et des informations d'accès. |
| **Trajets** | Gestion des itinéraires, horaires, statuts et caractéristiques des parcours. |

| Nom de la Table | Rôle et Description |
| --- | --- |
| **Véhicules** | Inventaire des véhicules de la flotte, caractéristiques et disponibilités. |
| **Réservations** | Enregistrement des demandes de réservation effectuées par les clients. |

### 2.3 Interface Client (Frontend)

- **Technologies proposées :** React.js, Vue.js (pour le web) ou Electron (pour une application desktop native).
- **Fonctionnalités clé :**
  - Interface utilisateur moderne, réactive et intuitive.
  - Visualisation dynamique et claire des trajets disponibles.
  - Module de réservation en ligne rapide.
  - Suivi en temps réel des véhicules et trajets (si applicable).

## 3. Spécifications Fonctionnelles

### 3.1 Gestion des Utilisateurs

- Modules d'inscription et de connexion sécurisés.
- Gestion des droits d'accès basée sur les rôles : **Admin**, **Chauffeur**, **Client**.

### 3.2 Gestion des Trajets

- Création, modification et suppression (CRUD) des trajets.
- Affectation dynamique des véhicules et des chauffeurs aux trajets planifiés.

### 3.3 Réservation et Suivi

- Prise de réservation par les clients en quelques clics.
- Processus de confirmation automatique et suivi de l'état de la réservation.

### 3.4 Reporting et Historique

- Tableaux de bord statistiques sur l'activité des trajets.
- Consultation et exportation de l'historique des réservations.

## 4. Sécurité et Conformité

- **Chiffrement :** Chiffrement des données sensibles au repos et en transit.
- **Authentification :** Procédures renforcées de gestion des sessions.

- **Protection contre les attaques :** Protections intégrées contre les injections SQL, les failles XSS et les requêtes intersites non autorisées (CSRF).

## 5. Stratégie de Déploiement

| Composant | Environnement de Déploiement | Outils / Méthode |
|---|---|---|
| **Serveur Node.js** | Windows Server | Gestionnaire de processus PM2 ou IIS Node |
| **Base de données** | Ubuntu Server (Linux) | Accès distant sécurisé (SSH / VPN) |
| **Application Client** | Navigateur Web / Desktop | Déploiement Web ou paquet Application Desktop |

## 6. Maintenance et Évolutivité

- **Documentation :** Rédaction d'une documentation technique et fonctionnelle complète.
- **Qualité du code :** Définition et exécution de tests unitaires et de tests d'intégration.
- **Évolutivité future :** Conception modulaire permettant l'intégration ultérieure de fonctionnalités complémentaires (modules de paiement en ligne, géolocalisation avancée, etc.).